

Webscraping using Python

Webscraping

- Download pages: requests.
- Parse pages: BeautifulSoup.
- References
 - <https://medium.freecodecamp.org/how-to-scrape-websites-with-python-and-beautifulsoup-5946935d93fe>
 - <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>



Webscraping

1. Download data using requests library.
2. Extract information using BeautifulSoup.

Example: Patenting data



Good conduct

- Check a website's Terms and Conditions before you scrape it. Usually, the data you scrape should not be used for commercial purposes.
- Do not request data from the website too aggressively with your program (also known as spamming), as this may break the website. One request for one webpage per second is good practice.
- The layout of a website may change from time to time, so make sure to revisit the site and rewrite your code as needed



1. Downloading from the web using request module

- simpler than urllib2
- requests: <http://docs.python-requests.org/>
- From the command line, run “pip install requests”.

Downloading from the web

- request.get fetches the url content.
- encoding is utf-8
- check that the response is ok
- 178,981 characters
- print first 250 characters

```
res = requests.get('http://www.gutenberg.org/cache/epub/1112/pg1112.txt')

type(res)
requests.models.Response

res.encoding
'utf-8'

res.status_code == requests.codes.ok
True

len(res.text)
178981

In [17]: print(res.text[:250])
The Project Gutenberg EBook of Romeo and Juliet, by William Shakespeare

This eBook is for the use of anyone anywhere at no cost and with
almost no restrictions whatsoever. You may copy it, give it away or
re-use it under the terms of the Proje
```

Downloading from the web

- Use exception handling

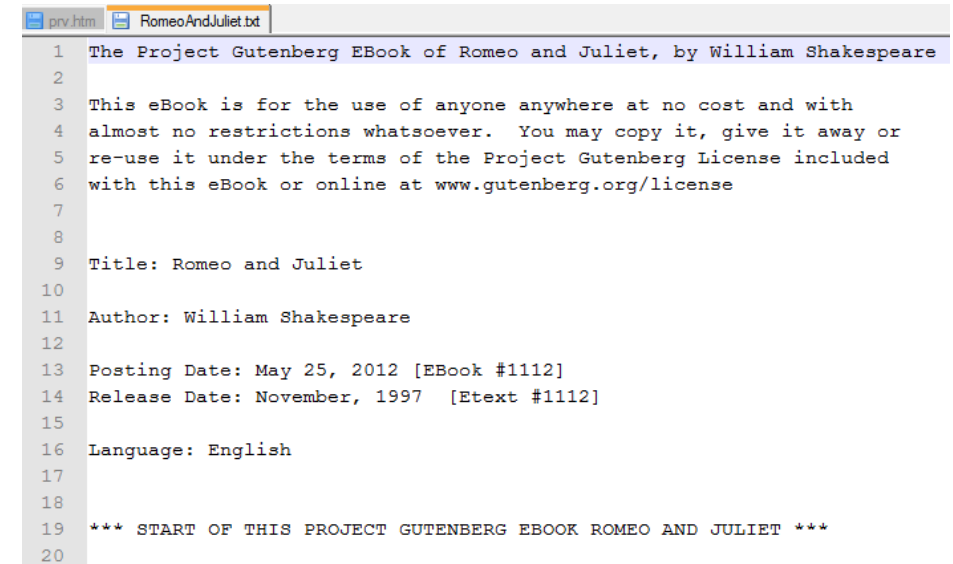
```
In [22]: try:
...:     res = requests.get('https://www.nek.su.se')
...:     res.raise_for_status()
...: except Exception as exc:
...:     print('There was a problem: %s' % (exc))
There was a problem: HTTPSConnectionPool(host='www.nek.su.se', port=443): Max retries exceeded with url: / (Caused by
NewConnectionError('<urllib3.connection.VerifiedHTTPSConnection object at 0x000002912AEF66A0>: Failed to establish a new
connection: [Errno 11001] getaddrinfo failed',))
```

Downloading from the web

- save raw file to disk (always do this for documentation)
- wb = write binary to save Unicode encoding.
- Save chunks of data iteratively to disk not to overload memory, (100000 thousand bytes chunks works).

```
playFile = open('RomeoAndJuliet.txt', 'wb')
for chunk in res.iter_content(100000):
    playFile.write(chunk)

playFile.close()
```



```
1 The Project Gutenberg eBook of Romeo and Juliet, by William Shakespeare
2
3 This eBook is for the use of anyone anywhere at no cost and with
4 almost no restrictions whatsoever. You may copy it, give it away or
5 re-use it under the terms of the Project Gutenberg License included
6 with this eBook or online at www.gutenberg.org/license
7
8
9 Title: Romeo and Juliet
10
11 Author: William Shakespeare
12
13 Posting Date: May 25, 2012 [EBook #1112]
14 Release Date: November, 1997 [Etext #1112]
15
16 Language: English
17
18
19 *** START OF THIS PROJECT GUTENBERG EBOOK ROMEO AND JULIET ***
20
```

Downloading from the web: PRV example

- Do a manual search and inspect the resulting address

Om databasen Kontakt Hjälp In English

PRV Svensk Patentdatabas

Här kan du söka bland svenska patent, offentliga patentansökningar och europeiska patent (EP-patent) som validerats i Sverige. Mer information finns i [Om databasen](#).

Sök i patentdatabasen

► Sök

Enkel sökning **Avancerad sökning** Klasssökning Tilläggsskydd (SPC)

► Min lista

I Avancerad sökning kan du kombinera olika sökkriterier och söka mer specifikt. Fälten är sammanbundna med ett logiskt "OCH". Se [instruktionsfilm](#) eller [söktips](#).

Länkar till andra tjänster och dokument (pdf) -öppnas i eget fönster:
[IPC-klasslistor \(WIPO\)](#)
[IPC-klasser \(på svenska\)](#)
[IPC-guiden](#)
[CPC-klasslistor \(EPO\)](#)
[CPC-klasslistor \(PDF\)](#)
[DPK \(tyska klasssystemet\)](#)
[IPC-DPK konkordanslista](#)
[Dokumentkoder](#)
[INID-koder](#)
[Authority Files](#)

Sök i all text: [Om textsökning](#)
☐ Sök bara i titel

Ansökningsnummer/publiceringsnummer: Alla [Om nummer](#)

Klass: [Om klasser](#)

Datum (fr.o.m - t.o.m): - Alla [Om datum](#)

Prioritetsnummer: [Om prioritetsnummer](#)

Sökande/innehavare: [Om sökande/innehavare](#)

Uppfinnare: [Om uppfinnare](#)

Ombud: [Om ombud](#)

Ort/postnummer: [Om ort/postnummer](#)

Tilläggssval vid ortsökning: [Om ortsökning](#)

Typ av kungörelse: Alla [Om kungörelse](#)

Publicerad i Svensk Patenttidning: År: 1999 Nr: - År: 1999 Nr: [Om Svensk Patenttidning](#)

Patentstatus: ☒ Alla patentansökningar och patent ☐ Enbart patent i kraft

Typ av ansökan/patent: ☒ Alla ☐ SE ☐ EP

Sökindex uppdaterat: 20201020 Senast inlagda kungörelser i Svensk Patenttidning: 2020-43

Rensa Sök

→ https://tc.prv.se/spd/search?hits=true&tab=2&tidar_tom=1999&tidar_from=1999&lang=sv&start=100&range=100

Downloading from the web

- PRV example

```
url = "http://was.prv.se/spd/search?hits=true&tidar_from=1999&tab=2&tidar_tom=1999&lang=sv&start=100&range=100"
```



PRV Svensk Patentdatabas

Om databasen Kontakt Hjälp In English

◀ Tillbaka till sök Ny sökning

Sökresultat ?

▶ Sök Exportera (upp till 500 träffar) Skriv ut

▶ Min lista Sökfråga (Avancerad sökning) Visa sökfråga ▼

Antal träffar: 14851 Träffar per sida: 100 ▼

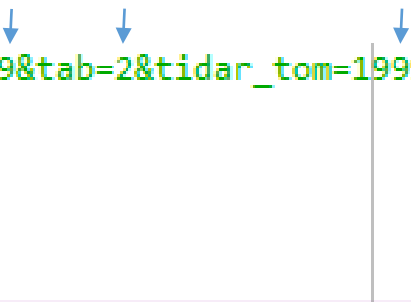
Länkar till andra tjänster och dokument (pdf) -öppnas i eget fönster:
[IPC-klasslistor \(WIPO\)](#)
[IPC-klasser \(på svenska\)](#)
[IPC-quiden](#)
[CPC-klasslistor \(EPO\)](#)
[CPC-klasslistor \(PDF\)](#)
[DPK \(tyska klasssystemet\)](#)
[IPC-DPK konkordanslista](#)

	Ansöknings-nummer	Publ.-nummer	Titel	Klass	Sökande	Ingivnings-datum
101	EP97922838.4	EP0885013	Nya indikationer för användande som läkemedel av multipotenta parapoximmunitetsindu	A61K39/275	Anton Mayr	1997-04-14
102	EP97921695.9	EP0832165	Hartshaltiga dispersionsklister	C08L93/04 C09J191/06 C09J193/04	Uzin Utz Aktiengesellschaft	1997-04-22
103	EP97921614.0	EP0892920	Förfarande och anordning för att prova en hållkroppplängd	G01N3/12	MANNESMANN Aktiengesellschaft	1997-04-11
104	EP97921612.4	EP0897542	Strömsensor, särskilt för små strömmar	G01R15/18 G01R19/20	Vacuumschmelze GmbH	1997-04-15

Downloading documents from the web

- requests to get the file and print it to file

```
url = "http://was.prv.se/spd/search?hits=true&tidar_from=1999&tab=2&tidar_tom=1999&lang=sv&start=100&range=100"
res = requests.get(url)
out_file = open('prv.htm', 'wb')
for chunk in res.iter_content(100000):
    out_file.write(chunk)
out_file.close()
```



Downloading from the web

- The file prv.htm on the hard disk.



Svensk Patentdatabas

Tillbaka till sök

Ny sökning

Sökresultat

PRV

Exportera (upp till 500 träffar)

Skriv ut

Sökfråga (Avancerad sökning)

Visa sökfråga

•

Länkar till andra tjänster och dokument (pdf)

- öppnas i eget fönster:

• [IPC-klasslistor \(WIPO\)](#)

• [IPC-klasser \(på svenska\)](#)

• [IPC-guiden](#)

• [CPC-klasslistor \(EPO\)](#)

• [CPC-klasslistor \(PDF\)](#)

• [DPK \(tyska\)](#)

• [klassystemet](#)

• [IPC-DPK konkordanslista](#)

Antal träffar: 14851

Träffar per sida:

100 ▼

	Ansöknings-nummer	Publ-nummer	Titel	Klass	Sökande	Ingivningsdatum
101	EP97922838.4	EP0885013	Nya indikationer för användande som läkemedel av multipotenta parapoximmunitetsinducerande medel från försvagade, icke immuniserade	1. A61K39/275	1. Anton Mayr	1997-04-14



2. Extract information using BeautifulSoup.

HTML tags

```
<!DOCTYPE html>
<html>
  <head>
  </head>
  <body>
    <h1> First Scraping </h1>
    <p> Hello World </p>
  </body>
</html>
```

This is the basic syntax of an HTML webpage. Every <tag> serves a block inside the webpage:

1. <!DOCTYPE html>: HTML documents must start with a type declaration.
2. The HTML document is contained between <html> and </html>.
3. The meta and script declaration of the HTML document is between <head> and </head>.
4. The visible part of the HTML document is between <body> and </body> tags.
5. Title headings are defined with the <h1> through <h6> tags.
6. Paragraphs are defined with the <p> tag.

Other useful tags include

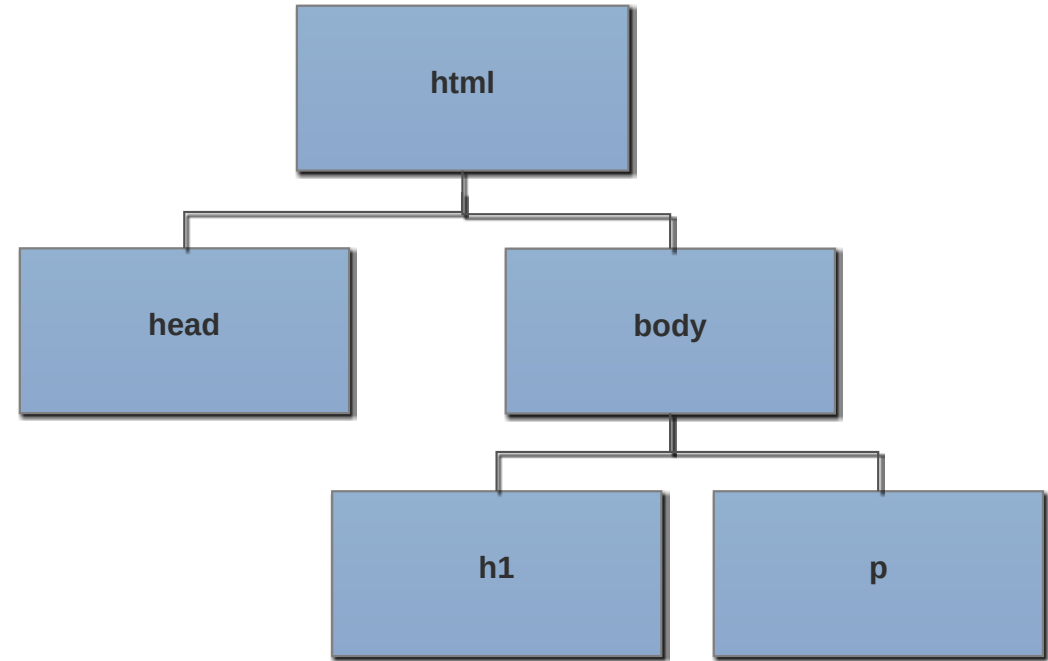
<a> for hyperlinks,
<table> for tables,
<tr> for table rows, and
<td> for table columns.

Also, HTML tags sometimes come with id or class attributes.

id attribute specifies a unique id for an HTML tag and the value must be unique within the HTML document. class attribute is used to define equal styles for HTML tags with the same class.

The HTML tree

```
<!DOCTYPE html>
<html>
  <head>
  </head>
  <body>
    <h1> First Scraping </h1>
    <p> Hello World </p>
  </body>
</html>
```



Inspecting a page

Goto a page, eg https://was.prv.se/spd/search?hits=true&tidar_from=1999&tab=2&tidar_tom=1999&lang=sv&start=100&range=100

Right-click and choose “view page source”

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN" "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
```

PRV

Svensk Patentdatabas



[Om databasen](#) [Kontakt](#) [Hjälp](#) [In English](#)

[Tillbaka till sök](#) [Ny sökning](#)

Sökresultat

[Exportera \(upp till 500 träffar\)](#) [Skriv ut](#)

Sökfråga (Avancerad sökning) [Visa sökfråga](#)

Antal träffar: 14851

Träffar per sida: 100

	Ansöknings-nummer	Publ.-nummer	Titel	Klass	Sökande	Ingivnings-datum
101	EP97922838.4	EP0885013	Nya indikationer för användande som läkemedel av multipotenta parapoximmunitetsinducerande medel från	A61K39/275	Anton Mayr	1997-04-14

[Länkar till andra tjänster och dokument \(pdf\)](#)
- öppnas i eget fönster:
[IPC-klasslistor \(WIPO\)](#)
[IPC-klasser \(på svenska\)](#)

```
<html
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:jsp="http://java.sun.com/JSP/Page"
  xmlns:c="http://java.sun.com/jsp/jstl/core">

  <head>

    <meta http-equiv="x-ua-compatible" content="IE=edge">
    <link rel="shortcut icon" type="image/x-icon" href="https://www.prv.se/...>
    <link rel="icon" type="image/x-icon" href="https://www.prv.se/...>
    <link href="/spd/jsp/common/top/top.css" rel="stylesheet">
    <link href="/spd/jsp/common/leftarea/leftarea.css" rel="stylesheet">
    <script type="text/javascript" src="/spd/jsincludes/jsincludes.js">
    <script type="text/javascript" src="/spd/jsincludes/jsincludes.js">
    <meta name="WT.ti" content="SVENSKPATENTDATABAS">

    <meta http-equiv="Content-Type" content="text/html">
    <link href="/spd/jsp/search/search.css" rel="stylesheet">
    <link href="/spd/css/print.css" media="print" rel="stylesheet">

    <title>Svensk Patentdatabas, Träfflista</title>
  </head>
  <body>
```

```
<tr>
  <td headers="nbr" style="font-style: italic;">
    101
  </td>
  <td headers="status">
    &nbsp;
  </td>
  <td headers="applicationno">
    <a
      id="putfocushere"
      href="/spd/patent?pl=lvOg0NiJXFhnn_6x9kySCg&p2=MosHn"
      >
      EP97922838.4
    </a>
  </td>
</tr>
```

Inspecting a page

- Right-click to view your browsers Inspector ("Inspect element" in Firefox).
- Identify the elements that you are interested in.
 - The number of patents is inside a few levels of HTML tags
 - The relevant table first has a row with hyperlinked headings, e.g. Ansöknings-nummer.

```
▼<div style="float: left; width: 200px;">  
  <span>Antal träffar: 14851</span>  
</div>
```

```
▼<table style="font-size: 60%; table-layout: fixed; width: 600px; border  
style:solid; border-bottom-width:1px;" cellpadding="0" border="0">  
  ▼<tbody>  
    ▼<tr>  
      <th id="nbr" style="width: 16px;"> ◀ </th> event  
      <th id="status" style="width: 14px;"> ◀ </th> event  
      ▼<th id="applicationno" class="" style="width: 81px;"> event  
        ▼<div class="hits_header">  
          ▶<div style="float: right;">⋮</div>  
          ▼<div>  
            <a href="/spd/search?sort=applicationno&hits=true&tidar_from=  
tidar_tom=1999&lang=sv">Ansöknings-nummer </a> event  
          </div>  
        </div>
```

Inspecting a page

- The following rows contains the data.

```
▼<tr class="">
  <td headers="nbr" style="font-style: italic;">1</td> event
  <td headers="status"> ◻ </td> event
  ▶<td headers="applicationno">...</td> event
  ▶<td headers="publicationno">...</td> event
  ▶<td headers="title" title="Anordning för reglering av flyghöjden hos styrbara flygande föremål">...</td> event
  ▶<td headers="classification" title=" G05D1/04, G05D1/06 ">...</td> event
  ▶<td headers="applicant" title=" Short Missile Systems Ltd ">...</td> event
  <td headers="filingdate">1976-01-09</td> event
</tr>
<!--create a search url-->
▼<tr class="secondrow">
  <td headers="nbr" style="font-style: italic;">2</td> event
  <td headers="status"> ◻ </td> event
  ▶<td headers="applicationno">...</td> event
  ▶<td headers="publicationno">...</td> event
  ▶<td headers="title" title="Multipelbaskomposition och sätt för dess framställning">...</td> event
  ▶<td headers="classification" title=" C06B25/20, C06B25/26, C06B45/10 ">...</td> event
```

Parsing with BeautifulSoup

```
url = "https://was.prv.se/spd/search?hits=true&tidar_from=1999&tab=2&tidar_tom  
out1 = 'prv.htm'  
out2 = 'prv.txt'
```

```
# First read the data from the web and save it to disk.
```

```
response = requests.get(url)  
out_file_raw = open(out1, 'wb')  
for chunk in response.iter_content(100000):  
    out_file_raw.write(chunk)  
out_file_raw.close()
```

```
# Then open the file and parse the information using the lxml parser.
```

```
infile = open(out1, 'rb')  
text=infile.read()  
soup = BeautifulSoup(text, 'lxml')
```

Find the table

```
# Find all tables. The first table is the one we want.
tables=soup.find_all('table')
table=tables[0]
```

```
rows=table.find_all('tr')
len(rows)
101
```

```
row=rows[0]
print(row)
```

```
"nbr" style="width: 16px;"> </th>
"status" style="width: 14px;"> </th>
```

```
# The second row contains the first data
row=rows[1]
print(row)
```

```
<tr>
<td headers="nbr" style="font-style: italic;">
```

```
101
```

```
</td>
```

```
# Determine the number of columns in the table
td_tags = row.find_all('td')
n_columns = len(td_tags)
```

n_columns	int	1	8
-----------	-----	---	---

Analyse a row

```
# Determine the number of columns in the table
td_tags = row.find_all('td')
n_columns = len(td_tags)
```



n_columns	int	1	8
-----------	-----	---	---

```
# Analyze the third tag
```

```
In [13]: tag2=td_tags[2]
...: tag2
```

```
Out[13]:
```

```
<td headers="applicationno">
<a href="/spd/patent?
p1=lvOg0NijXFnhn_6x9kySCg&p2=_MosHnGE6BU&a
100&start=100" id="putfocushere">
```

```
EP97922838.4
```

```
</a>
```

```
</td>
```

Tags in BS

- Tag object corresponds to HTML tag in the original document:

- `.name` returns the name

```
In [3]: tag2.name
```

```
Out[3]: 'td'
```

```
<td headers="applicationno">
```

```
<a
```

```
href="/spd/patent?p1=Z6EWPnkDrqFx4i
```

```
EP98113166.7
```

```
</a>
```

```
</td>
```

```
<td headers="publicationno">
```

Tags in BS

- Tag attributes:

- Attributes extracted as in dictionary

```
In [4]: tag2['headers']
```

```
Out[4]: ['applicationno']
```

```
<td headers="applicationno">
```

```
<a
```

```
href="/spd/patent?p1=Z6EWPnkDrqFx4i
```

- Access dictionary by attrs

```
In [5]: tag2.attrs
```

```
Out[5]: {'headers': ['applicationno']}
```

```
EP98113166.7
```

```
</a>
```

```
</td>
```

```
<td headers="publicationno">
```

Tags in BS

- Extract tag string:

- `get_text()` method returns all text in a single Unicode string

```
In [8]: tag2.get_text()
```

```
Out[8]: '\n\r\n
EP97922838.4\r\n
\n'
```

\r\n
 \r\n

\r\n
 \r\n

 headers="applicationno"> |

href="/spd/patent?p1=Z6EWPnkDrgFx4i

EP98113166.7

 $\langle \gamma \rangle$ |

- `get_text("|", strip=True)` strips whitespace

```
tag2.get_text("|", strip=True)
```

'EP97922838.4'

Printing the table

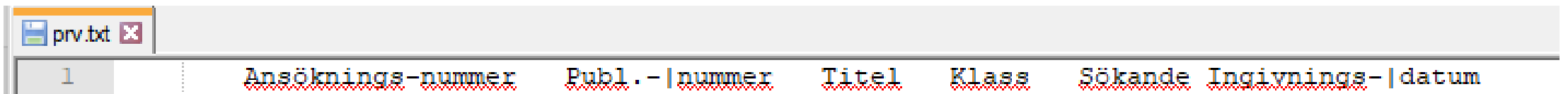
- Print the header row

```
# Find all tables. The first table is the one we want.
tables=soup.find_all('table')
table=tables[0]

# Create a list of rows
rows=table.find_all('tr')

# The first row contains the column headings
row=rows[0]

# Print the row names the first row of the output file
out_file_txt=open(out2,'w')
for th in row.find_all('th'):
    print(th.get_text("|", strip=True)+ "\t")
    out_file_txt.write(th.get_text("|", strip=True)+ "\t")
out_file_txt.write("\n")
```



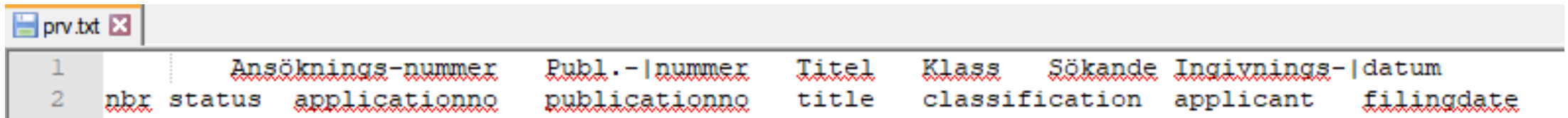
The screenshot shows a text editor window with a single file named 'prv.txt'. The editor displays a table header row with seven columns. The first column contains the number '1'. The other six columns contain the following text: 'Ansöknings-nummer', 'Publ.-|nummer', 'Titel', 'Klass', 'Sökande', and 'Ingivnings-|datum'. Each column header is underlined with a red dashed line.

1	<u>Ansöknings-nummer</u>	<u>Publ.- nummer</u>	<u>Titel</u>	<u>Klass</u>	<u>Sökande</u>	<u>Ingivnings- datum</u>
---	--------------------------	----------------------	--------------	--------------	----------------	--------------------------

Printing the table

- Alternative: print the heading attributes

```
# You can alternatively print the headers
row=rows[1]
for td in row.find_all('td'):
    header=re.sub(r'\W', '', str(td['headers']))
    out_file_txt.write(header+ "\t")
out_file_txt.write("\n")
```



The screenshot shows a text editor window titled 'prv.txt'. The content is a table with 8 columns and 2 rows. The first row contains Swedish labels for the columns, and the second row contains their English equivalents. The text is formatted with red underlines for each column header.

1	Ansöknings-nummer	Publ.-nummer	Titel	Klass	Sökande	Ingivnings-	datum
2	nbr status applicationno	publicationno	title	classification	applicant	filingdate	

Printing the table

- Print the table values

```
# Print the values in the table
for row in table.find_all('tr'):
    columns = row.find_all('td')
    for column in columns:
        out_file_txt.write(column.get_text("|", strip=True) + "\t")
    out_file_txt.write("\n")
out_file_txt.close()
```

prv.txt

1		Ansöknings-nummer	Publ.-nummer	Titel	
2	nbr	status	applicationno	publicationno	title
3					
4	101	EP97922838.4	EP0885013	Nya indikationer	
5	102	EP97921695.9	EP0832165	Hartshaltiga d	
6	103	EP97921614.0	EP0892920	Förfarande och	
7	104	EP97921612.4	EP0897542	Strömsensor, s	
8	105	EP97921189.3	EP0894312	System och and	

Data Editor (Browse) - [Untitled]

File

Edit

View

Data

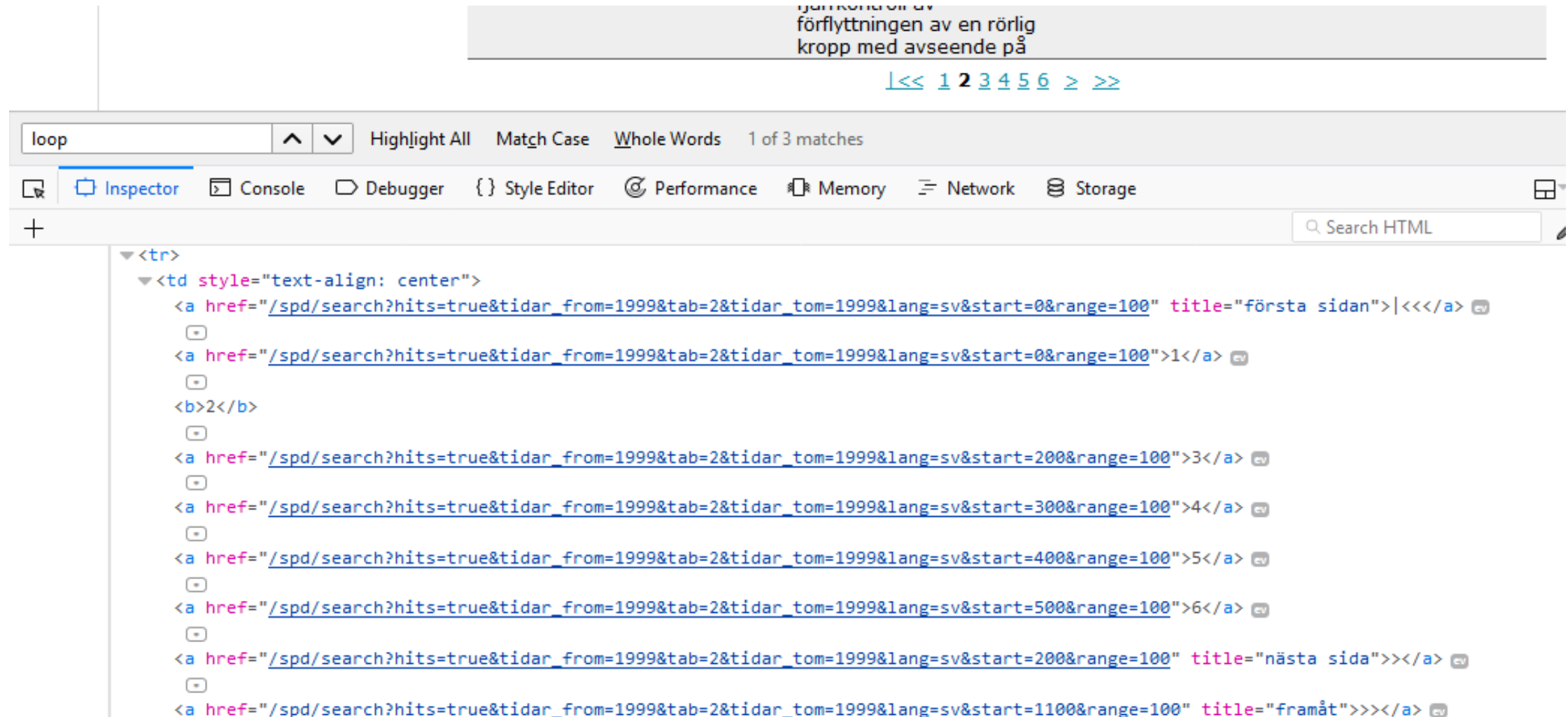
Tools

titel[16]

Asymmetriska bensoxantenfärgämnen

	v1	v2	ansökningsn~r	publnummer	
1	nbr	status	applicationno	publicationno	
2	101		EP97922838.4	EP0885013	
3	102		EP97921695.9	EP0832165	
4	103		EP97921614.0	EP0892920	
5	104		EP97921612.4	EP0897542	

Loop over web-pages to download



Extract the number of pages

- Using the span tag

```
# The number of observations are in a span tag
span_tags = soup.find_all('span')

#We can see that it is in the second tag
span_tags[0]
span_tags[1]
nobs=re.sub(r'\D', '',span_tags[1].get_text())
```

In [3]: nobs
Out[3]: '14851'

- Using regular expressions

```
# Extract number observations using regular expressions.
s1=re.compile('Antal träffar:\\r\\n *(\d+)*')
number_posts = s1.search(soup.get_text())[1]
number_pages=int(int(number_posts)/100)+1
```

In [5]: number_posts
Out[5]: '14851'

In [6]: number_pages
Out[6]: 149

Loop over web-pages to download

- Pattern start=0,100,200,...

```
<a href="/spd/search?hits=true&tidar_from=1999&tab=2&tidar_tom=1999&lang=sv&start=200&range=100">3</a>
```

```
<a href="/spd/search?hits=true&tidar_from=1999&tab=2&tidar_tom=1999&lang=sv&start=300&range=100">4</a>
```

```
<a href="/spd/search?hits=true&tidar_from=1999&tab=2&tidar_tom=1999&lang=sv&start=400&range=100">5</a>
```

```
for tab in range(1,number_pages):
    first=(tab-1)*100
    url = "https://was.prv.se/spd/search?hits=true&tidar_from=1999&tab=2&tidar_tom=1999&lang=sv&start="+ str(first) + "&range=100"
    out1 = 'prv' + str(tab) + '.htm'
    # First read the data from the web and save it to disk.
    response = requests.get(url)
    out_file_raw = open(out1,'wb')
    for chunk in response.iter_content(100000):
        out_file_raw.write(chunk)
    out_file_raw.close()
```

Loop over web-pages to parse

```
# Parse pages
outfile = 'prv_all.txt'

for tab in range(1,number_pages):
    in1 = 'prv' + str(tab) + '.htm'
    # Then open the file and parse the information using the lxml parser.
    infile = open(in1,'rb')
    text=infile.read()
    soup = BeautifulSoup(text, 'lxml')
```

Filling in a form

- A form tag has to specify
 - action, method, name
 - values
 - <https://classic.tvnews.vanderbilt.edu/tvn-search-advanced.pl>

Vanderbilt News Archive
Television

Home | Search | Requesting Videos | About the Archive | Institutional Subscriptions | Contact | login to my account

Home > Search > View result listing > View Item Details > Select > Checkout

TV-NewsSearch:
The Database of the Vanderbilt Television News Archive

Basic Search | Advanced Search | Browse by Date

Enter words to search:

Search

Word search ☒ Phrase search ☐ Boolean ☐

Limit search to a specific date (Optional): to

Network:

TV-NewsSearch:

The Database of the Vanderbilt Television News Archive

Basic Search

Advanced Search

Browse by Date

Enter words to search:

Search

Word search ☒ Phrase search ☐ Boolean ☐

Limit search to a specific date (Optional): January 1 2015 to December 1 2015

Network:

ALL Networks

```
<form style="display: inline; margin: 0;" action="https://classic.tvnews.vanderbilt.edu/tvn-processquery.pl" method="post" name="quicksearch">
  <input name="SID" value="20181005502775721" type="hidden">
  <input name="code" value="tvn" type="hidden">
  <table border="0">
    ...
    <td>
      <select name="Month" style="font-size: 90%; font-family: sans-serif; font-weight: bold;">
      <select name="Date" style="font-size: 90%; font-family: sans-serif; font-weight: bold;">
      <select name="Year" style="font-size: 90%; font-family: sans-serif; font-weight: bold;">
      to
      <select name="EndMonth" style="font-size: 90%; font-family: sans-serif; font-weight: bold;">
      <select name="EndDay" style="font-size: 90%; font-family: sans-serif; font-weight: bold;">
      <select name="EndYear" style="font-size: 90%; font-family: sans-serif; font-weight: bold;">
    </td>
```

Find the form tag

```
.....  
# Get the advanced search page.  
advanced_search_page = requests.get(advanced_link_href)  
advanced_soup = BeautifulSoup(advanced_search_page.text, 'lxml')  
  
#There is only one form. You can verify this using find_all('form')  
form=advanced_soup.find('form')  
form['name']  
'quicksearch'
```

```
# We need to specify the action.  
action = form['action']
```



Name	Type	Size	Value
action	str	1	https://classic.tvnews.vanderbilt.edu/tvn-processquery.pl

```
<form style="display: inline; margin: 0;" action="https://classic.tvnews.vanderbilt.edu/tvn-processquery.pl" method="post" name="quicksearch">
```

Find valid values to fill in form

```
In [44]: inputs = form.find_all('select')
```

```
In [45]: inputs
```

```
Out[45]:
```

```
[<select name="Month" style="font-size: 90%; font-family: sans-serif; font-weight: bold">
  <option></option>
  <option value="1">January</option>
  <option value="2">February</option>
  <option value="3">March</option>
  <option value="4">April</option>
  <option value="5">May</option>
  <option value="6">June</option>
  <option value="7">July</option>
  <option value="8">August</option>
  <option value="9">September</option>
  <option value="10">October</option>
  <option value="11">November</option>
  <option value="12">December</option>
</select>,
<select name="Date" style="font-size: 90%; font-family: sans-serif; font-weight: bold">
  <option></option>
  <option value="1">1</option>
  <option value="2">2</option>
```

Set the values to input in the form and post

```
# We need to specify the action.  
action = form['action']
```



Name	Type	Size	Value
action	str	1	https://classic.tvnews.vanderbilt.edu/tvn-processquery.pl

```
# Input values for dates to search  
start_year = '2014'; start_month = '1'; start_date = '1'  
end_year = '2014'; end_month = '1'; end_date = '31'
```

```
# payload initiates what to send to the page  
payload = {"Month": start_month, "Date": start_date, "Year": start_year,  
           "EndMonth": end_month, "EndDay": end_date, "EndYear": end_year}
```

```
# Fill in the form and post  
first_hit_page = requests.post(action, data=payload)
```

```
with open('test' + '.html', 'wb') as f:  
    f.write(first_hit_page.content)
```

Loop through results pages

- The results pages has a form called "nextpage" to get the next page. You can loop over results using this.
- Or by just looping over the hrefs.

Looping through the pages by form or href

20	01/01/2014	Upcoming Items (Studio: Lester Holt)	Evening News	NBC	05:41:30 pm	00:10																																		
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41
42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79			
80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113							
114																				Next																				

```
<form style="display: inline; margin: 0;" action="https://classic.tvnews.vanderbilt.edu/tvn-displayindex.pl" method="post" name="NextForm" >
<input type="hidden" name="SID" value="20181005502775721" />
<input type="hidden" name="code" value="tvn" />
<input type="hidden" name="pagenumber" value="3" />
<label accesskey="n" for="next">
<input type="image" name="submit" src="https://classic.tvnews.vanderbilt.edu/images/nextbutton.png" alt="Next Item" value="Next" id="next" />
</label>
</form>
```

```
<a href="https://classic.tvnews.vanderbilt.edu/tvn-displayindex.pl?SID=20181005502775721&pagenumber=2&code=tvn&" on
of results'; return true" onmouseout="self.status=' '; return true" title="Page 2 of results">2</a> ev
<a href="https://classic.tvnews.vanderbilt.edu/tvn-displayindex.pl?SID=20181005502775721&pagenumber=3&code=tvn&" on
of results'; return true" onmouseout="self.status=' '; return true" title="Page 3 of results">3</a> ev
<a href="https://classic.tvnews.vanderbilt.edu/tvn-displayindex.pl?SID=20181005502775721&pagenumber=4&code=tvn&" on
of results'; return true" onmouseout="self.status=' '; return true" title="Page 4 of results">4</a> ev
```

Loop by form

```
soup1 = BeautifulSoup(first_hit_page.text, 'lxml')
form=soup1.find('form')

# Now we need to click through the remaining pages.
# The session ID to keep stuff going when going forward
SID = re.search(r'SID=(\d*)', first_hit_page.text).group(1)
# Find number resulting pages, to loop over later.
number_of_pages = int(re.search(r"of <strong>(\d+)</strong>", first_hit_page.text).group(1))

# Get the action
action = form['action']

# Go through and save all the pages that was searched for in this session
for page_nr in range(2, number_of_pages + 1):
    loop_payload = {"SID" : SID, "code": "tvn", "pagenumber" : str(page_nr)}
    page = requests.post(action, data = loop_payload)
    file = 'page' + str(page_nr)
    with open(file + '.html', 'wb') as f:
        f.write(page.content)
```

Result



VANDERBILT Television News Archive

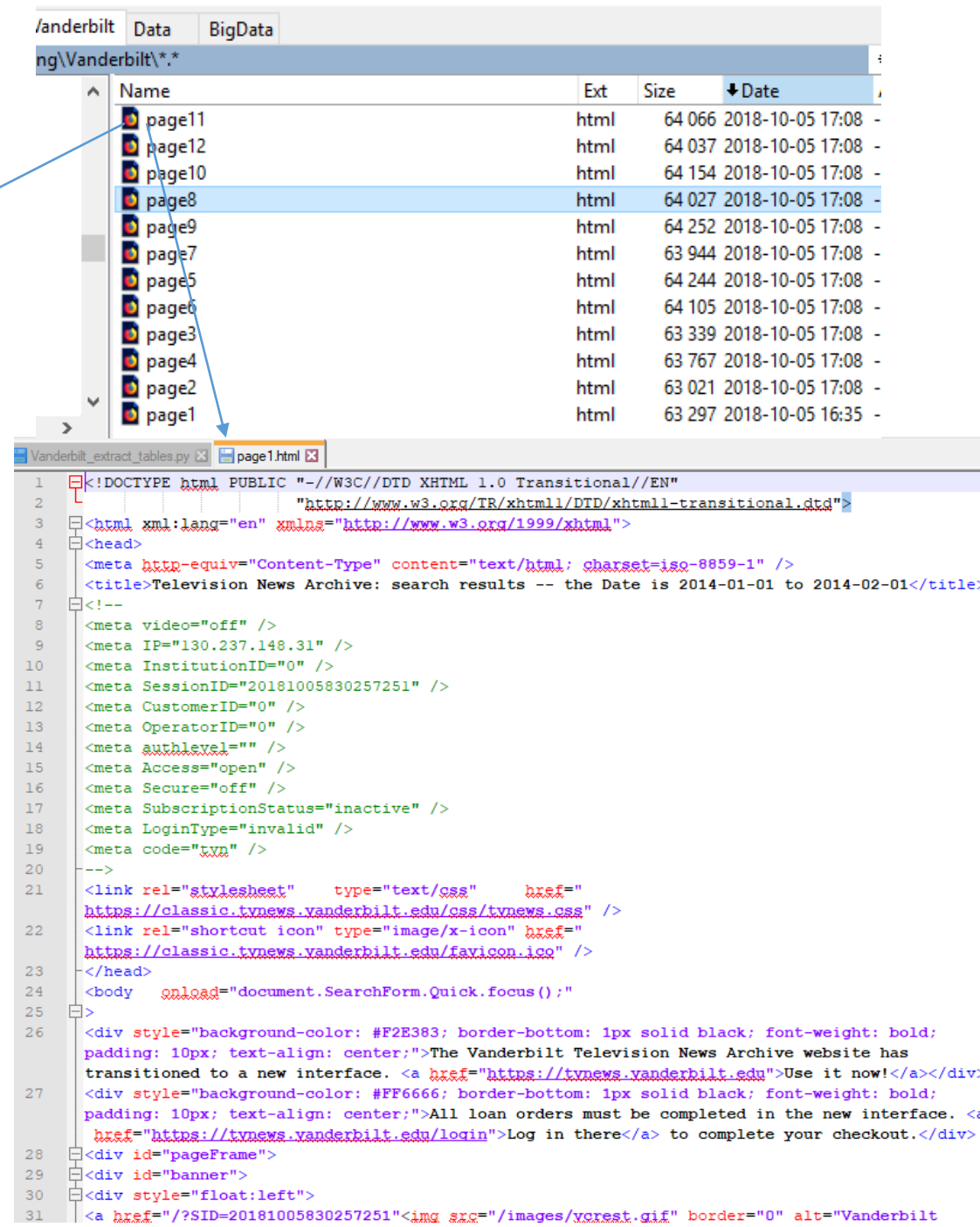
- [Home](#)
- [Search](#)
- [Requesting Videos](#)
- [About the Archive](#)
- [Institutional Subscriptions](#)
- [Contact](#)
- [login to my account](#)

found 2355 items where the Date is 2014-01-01 to 2014-02-01. Showing page 1 of 118.

1 [2](#) [3](#) [4](#) ... [118](#) Next Item search:

[Home](#) > [Search](#) > [View result listing](#) > View Item Details > Select > Checkout

No.	Date	Headline	Video	Type	Network	Begin	Length
1	02/01/2014	Good Night		Evening News	CNN	06:01:50 pm	00:10
2	02/01/2014	Good Night		Evening News	NBC	05:59:10 pm	00:10
3	02/01/2014	Good Night		Evening News	CBS	05:58:50 pm	00:10
4	02/01/2014	Good Night		Evening News	ABC	05:58:40 pm	00:10
5	02/01/2014	Puppy Bowl		Evening News	NBC	05:57:00 pm	02:10
6	02/01/2014	Pro Football / Super Bowl / The Phrase		Evening News	ABC	05:56:40 pm	02:00
7	02/01/2014	Football / Super Bowl / Puppy Predictors		Evening News	NBC	05:56:30 pm	00:30
8	02/01/2014	Football / Super Bowl / Fleming		Evening News	CBS	05:56:20 pm	02:30



Parse

```
7
8 import os, re
9 from bs4 import BeautifulSoup
10
11
12 # datadir is where I want to save my output
13 indir = 'e:/c_old/David/Projects/Old/Eisensee/iMacros/Python/Out'
14 outfile = 'e:/c_old/David/Projects/Old/Eisensee/iMacros/Python/Data/VN2014_2018.txt'
15
16
17 os.chdir(indir)
18 filelist = os.listdir(indir)
19
20 first=1
21
22 for file in filelist:
23     infile=open(file,'rb')
24     text=infile.read()
25     soup=BeautifulSoup(text, 'lxml')
26
27     # Find all tables. The second table is the one we want.
28     tables=soup.find_all('table')
29     table=tables[1]
30
31     # Create a list of rows
32     rows=table.find_all('tr')
33
34     if first==1:
35         # The first row contains the column headings
36         row=rows[0]
37         # Print the row names the first row of the output file
38         out_file_txt=open(outfile,'w', encoding='utf-8')
39         for th in row.find_all('th'):
40             print(th.get_text("|", strip=True)+ "\t")
41             out_file_txt.write(th.get_text("|", strip=True)+ "\t")
42         out_file_txt.write("\n")
43         first=0
44
45     # Print the values in the table
46     for row in table.find_all('tr'):
47         columns = row.find_all('td')
48         for column in columns:
49             value=column.get_text("|", strip=True)
50             value=re.sub('\t',' ',value)
51             out_file_txt.write(value + "\t")
52         out_file_txt.write("\n")
53     out_file_txt.close()
```

Data Editor (Browse) - [Untitled]

File Edit View Data Tools



no[1]		601					
	no	date	headline	type	network	begin	length
1	601	09/05/2018	Upcoming Items & COMMERCIALS: Vraylar; Safelite; ...	Evening News	ABC	05:49:10 pm	03:30
2	602	09/05/2018	Sports: Nike & Kaepernick	Evening News	ABC	05:47:50 pm	01:20
3	603	09/05/2018	Upcoming Items & COMMERCIALS: OxiClean; Publisher...	Evening News	ABC	05:45:10 pm	02:40
4	604	09/05/2018	Senate / Social Media Hearings	Evening News	ABC	05:44:40 pm	00:30
5	605	09/05/2018	New York, New York / Airport Illness	Evening News	ABC	05:43:00 pm	01:40
6	606	09/05/2018	Tropical Storm Gordon	Evening News	ABC	05:40:50 pm	02:10
7	607	09/05/2018	Supreme Court / Kavanaugh Nomination	Evening News	ABC	05:37:30 pm	03:20
8	608	09/05/2018	Trump / "New York Times" Letter / Woodward's Book	Evening News	ABC	05:31:40 pm	05:50
9	609	09/05/2018	Preview/Introduction David Muir (New York)	Evening News	ABC	05:30:00 pm	01:40
10	610	09/04/2018	Good Night	Evening News	CNN	08:00:10 pm	00:10
11	611	09/04/2018	Facebook	Evening News	CNN	07:59:50 pm	00:20
12	612	09/04/2018	Tropical Storm Gordon	Evening News	CNN	07:57:40 pm	02:10
13	613	09/04/2018	Upcoming Items & COMMERCIALS: "Cuomo Prime Time";...	Evening News	CNN	07:52:20 pm	05:20
14	614	09/04/2018	Trump / Mueller Investigation	Evening News	CNN	07:47:50 pm	04:30
15	615	09/04/2018	Upcoming Items & COMMERCIALS: Lexus; Banquet; Ace...	Evening News	CNN	07:44:40 pm	03:10
16	616	09/04/2018	Supreme Court / Kavanaugh Nomination / Whitehouse...	Evening News	CNN	07:31:50 pm	12:50
17	617	09/04/2018	Upcoming Items & COMMERCIALS: MiraLAX; Hotwire; ML...	Evening News	CNN	07:28:10 pm	03:40
18	618	09/04/2018	Trump / Mueller Investigation	Evening News	CNN	07:20:20 pm	07:50
19	619	09/04/2018	Upcoming Items & COMMERCIALS: "White Boy Rick"; F...	Evening News	CNN	07:17:00 pm	03:20

Selenium

- Initially a tool created to test a website's behavior, quickly became a general web browser automation tool.
- Starts and implements actions in a web browser.

References

- <https://selenium-python.readthedocs.io/navigating.html>
- <https://iqss.github.io/dss-webscrape/>

Installation

- `pip install selenium`
- Install driver (web browser).
 - Download driver and set path to executable in code.

Selenium or Requests

Requests

Great for handling simple webscraping, e.g. looping over pages.

Handles GET or POST forms in a simple and transparent way.

Cannot activate JavaScript functions

Selenium

Mimicks an actual web browser

Is able to activate JavaScript on a GET request

Harder to detect as bot.

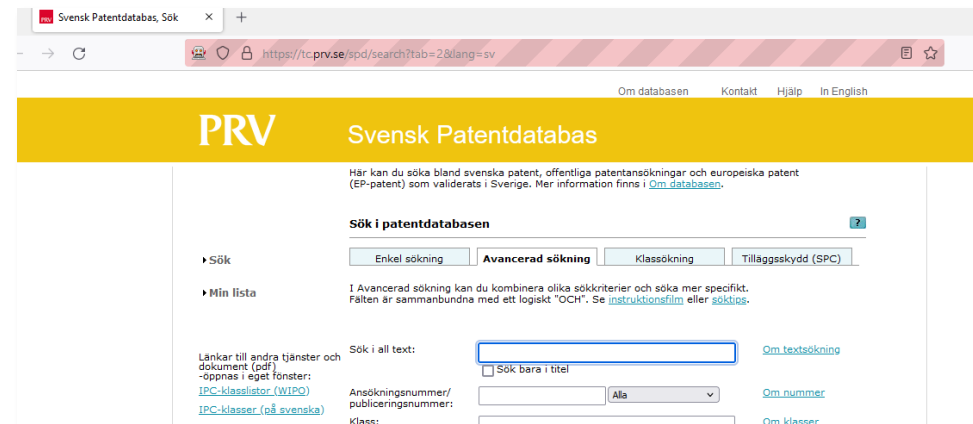
Good at pinpointing HTML tags to click, but a bit less transparent.

Selenium

Create selenium web browser object and open web browser window

```
5 from selenium import webdriver
```

```
14 DRIVER_PATH = 'C:/Users/dstro/AppData/Local/firefox/geckodriver.exe'  
15 driver = webdriver.Firefox(executable_path=DRIVER_PATH)  
16 driver.get("https://tc.prv.se/spd/search?tab=2&lang=sv")
```



Finding web page elements

Find element by id, name, xpath, css, class name, tag name,...

```
element= driver.find_element(By.ID, "passwd-id")
```

Returns element or set of elements with matching id attribute values.

```
<input type="text" name="passwd" id="passwd-id" />
```

can be found using:

```
element= driver.find_element(By.ID, "passwd-id")
```



```
(By.NAME, "passwd")
```

```
(By.XPATH, "//input[@id='passwd-id']")
```

```
(By.CSS_SELECTOR, "input#passwd-id")
```

Finding web page elements

```
find_element(By.ID, "id")  
find_element(By.NAME, "name")  
find_element(By.XPATH, "xpath")  
find_element(By.LINK_TEXT, "link text")  
find_element(By.PARTIAL_LINK_TEXT, "partial link text")  
find_element(By.TAG_NAME, "tag name")  
find_element(By.CLASS_NAME, "class name")  
find_element(By.CSS_SELECTOR, "css selector")
```

Enter text into text field:

```
22  # The element is located by its name attribute using By.NAME as its first parameter.
23  year_from=driver.find_element(By.NAME, "tidar_from")
24  # First clear any pre-populated text in the input field
25  year_from.clear()
26  # Then set the from year.
27  year_from.send_keys("1999")
28
29  #Set end year
30  year_to=driver.find_element(By.NAME, "tidar_tom")
31  year_to.clear()
32  year_to.send_keys("1999")
33
34  #Click on search
35  driver.find_element(By.XPATH, "//input[@value=\"Sök\"]").click()
```

Dropdown alternatives, save, click next.

```
--
37 # Find all options for the range dropdown menu.
38 dropdown = driver.find_element(By.NAME, "range")
39
40 all_options = dropdown.find_elements(By.TAG_NAME, "option")
41 for option in all_options:
42     print(option.get_attribute("value"))
43
44 #Set range to 100
45 dropdown.find_element(By.XPATH, "//option[. = '100']").click()
46
47 # Get source code
48 html = driver.page_source
49
50 # Print to file
51 out_file = open('prv_selenium.htm',"w", encoding="utf-8")
52 out_file.write(html)
53 out_file.close()
54
55 # Wait for page to load
56 time.sleep(2)
57
58 # Click on next page
59 driver.find_element(By.LINK_TEXT, "2").click()
60
```

Other useful commands

To move backward and forward in your browser's history:

```
driver.forward()  
driver.back()
```

```
# Now set the cookie. This one's valid for the entire domain  
cookie = {'name' : 'foo', 'value' : 'bar'}  
driver.add_cookie(cookie)
```

JavaScript: browser view

TELEVISION
Vanderbilt News Archive

ABOUT EXPLORE REQUEST SPONSORS CONTACT SUPPORT THE ARCHIVE SEARCH

EXPLORE Choose a date and explore events Explore by collection instead?

JAN 1 1976 SUBMIT

INTRODUCTION HARRY REASONER (NEW YORK CITY) #39763
ABC Evening News for 1976-01-01 Duration: 00:00:10

1776-1976 / LIBERTY BELL #39764
ABC Evening News for 1976-01-01 Reporter(s): Gregory, Bettina; Reasoner, Harry
Duration: 00:02:10

Live Chat with the Libraries

Inspector Console Debugger 2

Search HTML

```
<body class="page scheme-dark search calendar">
  <script>
  </script>
  <div id="wrap-header" class="wrap">
  </div>
  <div class="wrap">
    <!--
    <div class="alert alert-icon fade in" style="margin-top:7em;margin-bottom:-6em;">
      <a href="#" class="close" data-dismiss="alert">&times;</a>
      <span>Message here</span>
    </div>
    -->
    <div class="box slim first-box">
    </div>
    <div>
    </div>
    <div class="box calendar-select">
    </div>
    <section id="errors"></section>
    <section id="results-list" class="results-list box">
      ::before
      <ul class="results-list">
        ::before
        <li class="search-result" data-id="0">
          <div class="search-info">
            <h4 class="title">
            </h4>
            <p class="result-id">
            </p>
            <ul class="list-unstyled list-horizontal description">
              <li class="source">
              </li>
              <li class="duration">
                <strong>Duration:</strong>
                00:00:10
              </li>
            </ul>
          </div>
          <ul class="list-unstyled list-horizontal interactions">
          </ul>
        </li>
        <li class="search-result" data-id="1">
        </li>
        <li class="search-result" data-id="2">
        </li>
        <li class="search-result" data-id="3">
        </li>
        <li class="search-result" data-id="4">
        </li>
        <li class="search-result" data-id="5">
        </li>
        <li class="search-result" data-id="6">
        </li>
      </ul>
    </section>
  </div>
  </div>
```

But source does
not contain table info

```
230 <section class="results-list box" id="results-list">
231   <script type="text/javascript">
232     var currentOrderIds = [];
233     var currentUserId = "";
234   </script>
235 </section>
```

Execute JavaScript in the current window

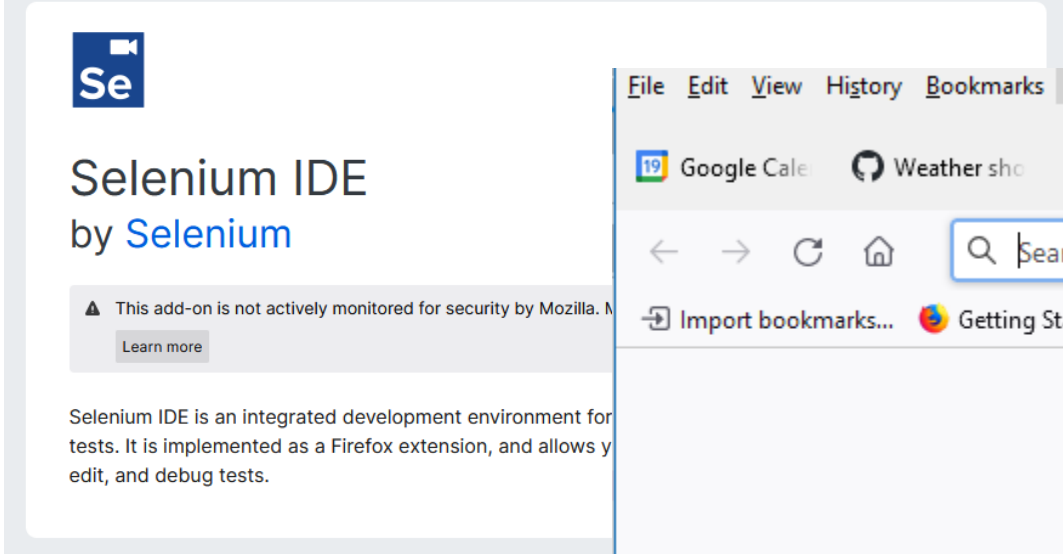
To extract information. We first need to execute the JavaScript.

```
# HTML from element with some JavaScript
element = driver.find_element(By.ID, "results-list")
time.sleep(5)
htmli = driver.execute_script("return arguments[0].innerHTML;", element)

# Extract table using bs
soup1 = BeautifulSoup(htmli, 'lxml')
```

We can the save text or extract table info using bs.
Strange syntax for the htmli argument, but it works.

 <https://addons.mozilla.org/en-US/firefox/addon/selenium-ide/>



Selenium IDE for Firefox/Chrome

Browse to start page

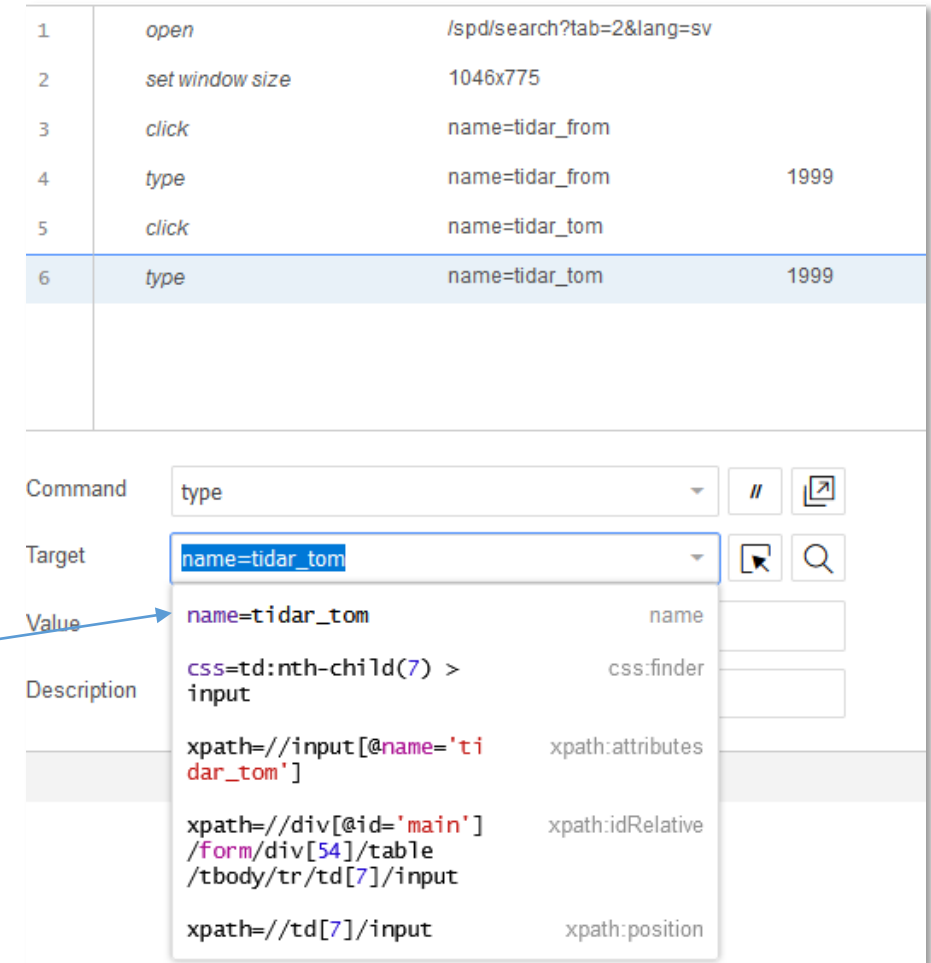
Click on Se icon  in browser

Record a new test in a new project.

Name and provide start page url.

Start recording commands.

(List of alternative location identifyers)



The screenshot displays the Selenium IDE interface. At the top, a table lists recorded commands:

Step	Command	Value	Value
1	open	/spd/search?tab=2&lang=sv	
2	set window size	1046x775	
3	click	name=tidar_from	
4	type	name=tidar_from	1999
5	click	name=tidar_tom	
6	type	name=tidar_tom	1999

Below the table, the 'Command' field is set to 'type'. The 'Target' dropdown menu is open, showing a list of alternative location identifiers for the element 'name=tidar_tom':

- name=tidar_tom (name)
- css=td:nth-child(7) > input (css:finder)
- xpath=//input[@name='tidar_tom'] (xpath:attributes)
- xpath=//div[@id='main']/form/div[54]/table/tbody/tr/td[7]/input (xpath:idRelative)
- xpath=//td[7]/input (xpath:position)

Selenium IDE for Firefox/Chrome

Stop recording

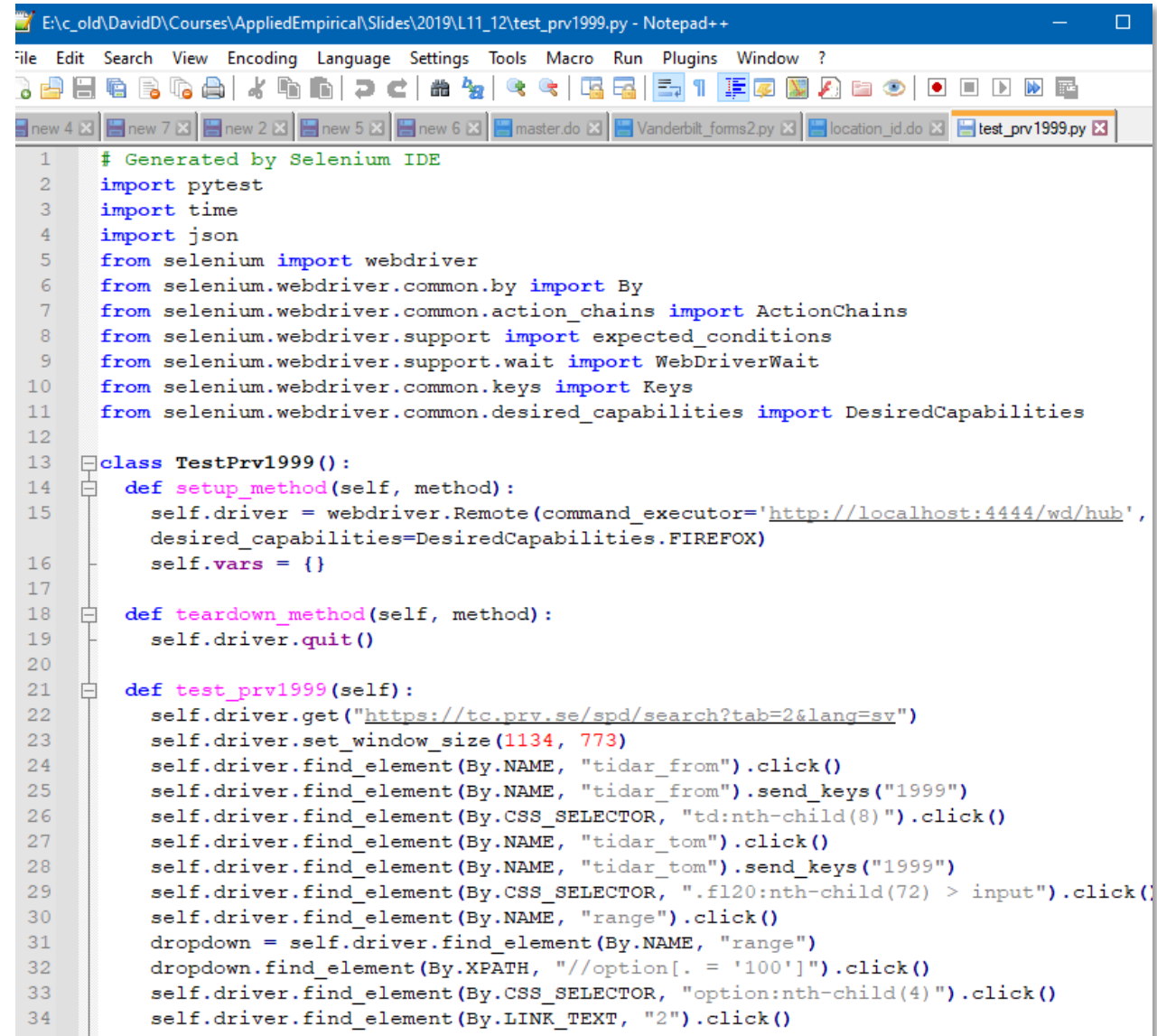
Name task.



Right-click on script
to export to python.

Save .py file.

Edit commands for your
scraper.

A screenshot of a Notepad++ window. The title bar shows the file path 'E:\c_old\DavidD\Courses\AppliedEmpirica\Slides\2019\L11_12\test_prv1999.py - Notepad++'. The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and ?. The toolbar contains various icons for file operations and editing. The tab bar shows several open files, with 'test_prv1999.py' selected. The main text area contains Python code generated by Selenium IDE. The code starts with a comment '# Generated by Selenium IDE' and includes imports for pytest, time, json, selenium.webdriver, and various Selenium actions and waits. It defines a class 'TestPrv1999()' with 'setup_method', 'teardown_method', and 'test_prv1999' methods. The 'test_prv1999' method contains a series of Selenium commands like 'get', 'set_window_size', 'find_element', 'click', and 'send_keys' to interact with a web page.

```
1 # Generated by Selenium IDE
2 import pytest
3 import time
4 import json
5 from selenium import webdriver
6 from selenium.webdriver.common.by import By
7 from selenium.webdriver.common.action_chains import ActionChains
8 from selenium.webdriver.support import expected_conditions
9 from selenium.webdriver.support.wait import WebDriverWait
10 from selenium.webdriver.common.keys import Keys
11 from selenium.webdriver.common.desired_capabilities import DesiredCapabilities
12
13 class TestPrv1999():
14     def setup_method(self, method):
15         self.driver = webdriver.Remote(command_executor='http://localhost:4444/wd/hub',
16                                       desired_capabilities=DesiredCapabilities.FIREFOX)
17         self.vars = {}
18
19     def teardown_method(self, method):
20         self.driver.quit()
21
22     def test_prv1999(self):
23         self.driver.get("https://tc.prv.se/spd/search?tab=2&lang=sv")
24         self.driver.set_window_size(1134, 773)
25         self.driver.find_element(By.NAME, "tidar_from").click()
26         self.driver.find_element(By.NAME, "tidar_from").send_keys("1999")
27         self.driver.find_element(By.CSS_SELECTOR, "td:nth-child(8)").click()
28         self.driver.find_element(By.NAME, "tidar_tom").click()
29         self.driver.find_element(By.NAME, "tidar_tom").send_keys("1999")
30         self.driver.find_element(By.CSS_SELECTOR, ".fl20:nth-child(72) > input").click()
31         self.driver.find_element(By.NAME, "range").click()
32         dropdown = self.driver.find_element(By.NAME, "range")
33         dropdown.find_element(By.XPATH, "//option[. = '100']").click()
34         self.driver.find_element(By.CSS_SELECTOR, "option:nth-child(4)").click()
35         self.driver.find_element(By.LINK_TEXT, "2").click()
```

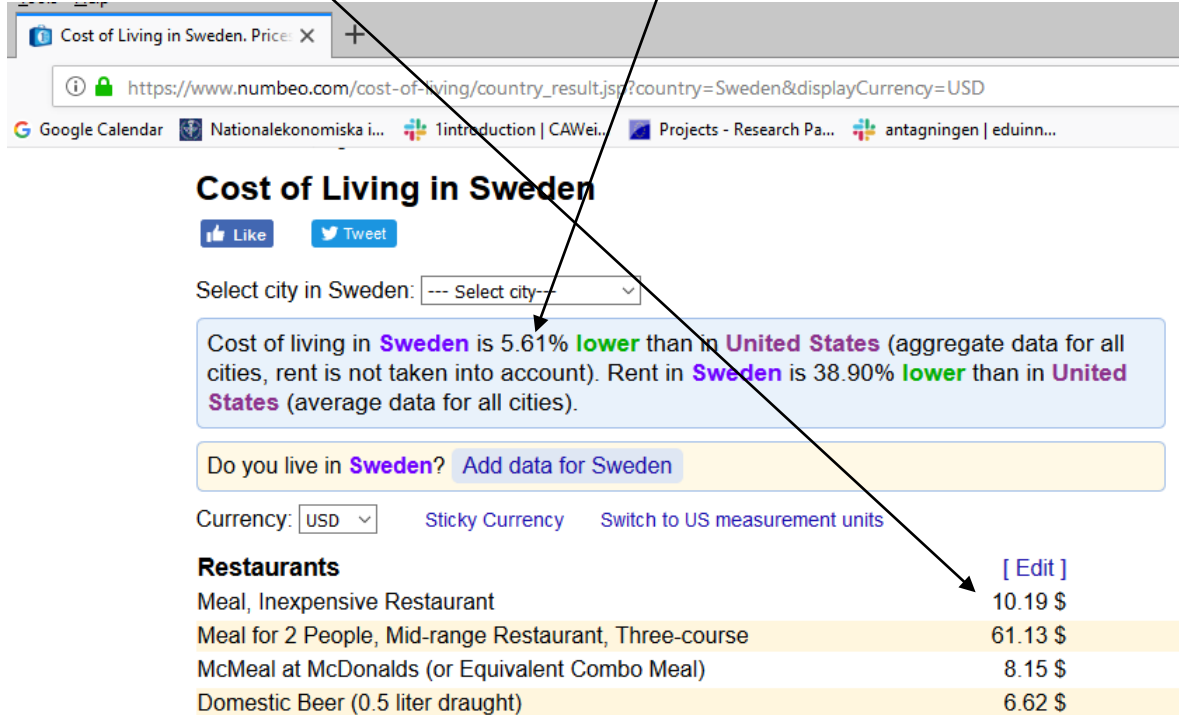
Other features

Headless mode: no visible browser, runs silently in background.

WebDriverWait: wait until certain conditions are met (e.g. certain element loaded)

Task 9

[Numbeo](https://www.numbeo.com) is a website where users supply price information in different locations. The task is to get the cost of living in USD(\$) for all countries. Get the name of each product category, the price of the product in USD. Also get the cost of living relative to the United States. Save this together with the country names in a csv file.



The screenshot shows the 'Cost of Living in Sweden' page on Numbeo. The URL is https://www.numbeo.com/cost-of-living/country_result.jsp?country=Sweden&displayCurrency=USD. The page displays the following information:

- Cost of Living in Sweden**
- Like and Tweet buttons
- Select city in Sweden: --- Select city ---
- Cost of living in Sweden is 5.61% lower than in United States (aggregate data for all cities, rent is not taken into account). Rent in Sweden is 38.90% lower than in United States (average data for all cities).
- Do you live in Sweden? Add data for Sweden
- Currency: USD (Sticky Currency, Switch to US measurement units)
- Restaurants**
- [Edit]

Meal, Inexpensive Restaurant	10.19 \$
Meal for 2 People, Mid-range Restaurant, Three-course	61.13 \$
McMeal at McDonalds (or Equivalent Combo Meal)	8.15 \$
Domestic Beer (0.5 liter draught)	6.62 \$

1. Use the request module to download and save as .htm <https://www.numbeo.com/cost-of-living/> .
2. Load the file into a BeautifulSoup object. Find all possible country input values. You can see all parameters in the URL, for example, https://www.numbeo.com/cost-of-living/country_result.jsp?country=Sweden&displayCurrency=USD. This means that you can just loop over country names in the url.
3. Download and save one country file. Load the file into a BeautifulSoup object. Find the relevant table. Print the table headings and values, together with the country name to a file. Then use regular expressions to extract the cost of living relative to the Sweden. Add this to the printed table.
4. Loop over all countries and do step 3. Note that URLs never include whitespaces; therefore, check what happens when a country name consists of multiple words.
5. Read all files into Stata.

The python code should be submitted along with the requested output.